

Лабораторный практикум. Часть 5

Лекция

Флаг переноса

Загрузить число 1 в регистр BX, число FFFFh в регистр AX:

```
-r bx
BX 0000
:1
-r ax
AX 0000
:ffff
-r
AX=FFFF BX=0001 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B0E ES=0B0E SS=0B0E CS=0B0E IP=0100 NU UP EI PL NZ NA PO NC
```

Записать команду «add ax, bx» в память:

```
-a 100
0B0E:0100 add ax, bx
0B0E:0102
-r
AX=FFFF BX=0001 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B0E ES=0B0E SS=0B0E CS=0B0E IP=0100 NU UP EI PL NZ NA PO NC
0B0E:0100 01D8 ADD AX,BX
```

Трассировать команду:

```
-t
AX=0000 BX=0001 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B0E ES=0B0E SS=0B0E CS=0B0E IP=0102 NU UP EI PL ZR AC PE CY
0B0E:0102 B43E MOV AH,3E
```

Установить 100h d IP и прибавить единицы к нулю в AX:

```
-r ip
IP 0102
:100
-t
AX=0001 BX=0001 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B0E ES=0B0E SS=0B0E CS=0B0E IP=0102 NU UP EI PL NZ NA PO NC
0B0E:0102 B43E MOV AH,3E
```

Циклический сдвиг.

Записать команду «rcl bl, 1» в память и число B7h в BX:

```
-a 100
0B0E:0100 rcl bl, 1
0B0E:0102
-r bx
BX 0001
:b7
-r ip
IP 0102
:100
-r
AX=0001 BX=00B7 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B0E ES=0B0E SS=0B0E CS=0B0E IP=0100 NU UP EI PL NZ NA PO NC
0B0E:0100 D0D3 RCL BL,1
```

```
-a 100
0B0E:0100 mov dl, 00
0B0E:0102 rcl bl, 1
0B0E:0104 adc dl, 30
0B0E:0107 mov ah, 02
0B0E:0109 int 21
0B0E:010B int 20
0B0E:010D
```

Организация циклов.

Вывод четырех звездочек:

```
-a 100
0B0E:0100 mov ah, 02
0B0E:0102 mov dl, 2a
0B0E:0104 mov cx, 4
0B0E:0107 int 21
0B0E:0109 loop 107
0B0E:010B int 20
0B0E:010D
-g
****
Программа завершилась нормально
```

ЛАБОРАТОРНАЯ РАБОТА №4

```
-a 100
0B05:0100 mov ah, 02
0B05:0102 mov dl, 28
0B05:0104 int 21
0B05:0106 mov dl, 42
0B05:0108 int 21
0B05:010A mov dl, 50
0B05:010C int 21
0B05:010E mov dl, 29
0B05:0110 int 21
0B05:0112 mov dl, 20
0B05:0114 int 21
0B05:0116 mov dl, 3d
0B05:0118 int 21
0B05:011A mov dl, 20
0B05:011C int 21
```

```
0B05:011E mov cx, 10
0B05:0121 mov bp, 1f48
0B05:0124 mov dl, 00
0B05:0126 rcl bp, 1
0B05:0128 adc dl, 30
0B05:012B int 21
0B05:012D loop 124
0B05:012F int 21
```

```
0B05:0131 mov dl, 0a
0B05:0133 int 21
0B05:0135 mov dl, 0d
0B05:0137 int 21
```

```
0B05:0139 mov dl, 28
0B05:013B int 21
0B05:013D mov dl, 41
0B05:013F int 21
0B05:0141 mov dl, 58
0B05:0143 int 21
0B05:0145 mov dl, 29
0B05:0147 int 21
0B05:0149 mov dl, 20
0B05:014B int 21
0B05:014D mov dl, 3d
0B05:014F int 21
0B05:0151 mov dl, 20
0B05:0153 int 21
```

```

0B05:0155 mov ax, d892
0B05:0158 xor cx, cx
0B05:015A mov bx, 2
0B05:015D xor dx, dx
0B05:015F div bx
0B05:0161 push dx
0B05:0162 inc cx
0B05:0163 test ax, ax
0B05:0165 jnz 15d
0B05:0167 mov ah, 02
0B05:0169 pop dx
0B05:016A add dl, 30
0B05:016D int 21
0B05:016F loop 169
0B05:0171 mov dl, 30
0B05:0173 int 20

```

```

-g
<BP> = 00011111010010000
<AX> = 1101100010010010
Программа завершилась нормально

```

Проверка вывода из DOS

```

C:\DOCUME~1\0545~1>Bin1
<BP> = 00011111010010000
<AX> = 1101100010010010

```

Лабораторный практикум. Часть 6

Лекция

Флаги состояния.

Отрицательный флаг знака:

```

-a 100
0B11:0100 sub ax, bx
0B11:0102
-t
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B11 ES=0B11 SS=0B11 CS=0B11 IP=0102  NU UP EI PL ZR NA PE NC
0B11:0102 B402          MOV     AH,02

```

Положительный знак состояния:

```

-r ax
AX 0000
:1
-r ip
IP 0102
:100
-t
AX=0001 BX=0000 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B11 ES=0B11 SS=0B11 CS=0B11 IP=0102  NU UP EI PL NZ NA PO NC
0B11:0102 B402          MOV     AH,02

```

Загрузить одно число F5h в регистры BX и AX:

```

-r ax
AX 0001
:f5
-r bx
BX 0000
:f5
-a 100
0B11:0100 cmp ax, bx
0B11:0102
-r ip
IP 0102
:100
-t

AX=00F5 BX=00F5 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B11 ES=0B11 SS=0B11 CS=0B11 IP=0102  NV UP EI PL ZR NA PE NC
0B11:0102 B402          MOV     AH,02

```

Команды условного перехода.

```

-a 100
0B11:0100 sub al, 01
0B11:0102 jnz 100
0B11:0104 int 20
0B11:0106
-g

```

Вывод на экран одной шестнадцатеричной цифры.

```

-r bx
BX 0000
:f
-a 100
0B11:0100 mov dl, bl
0B11:0102 cmp dl, 09
0B11:0105 ja 10c
0B11:0107 add dl, 30
0B11:010A jmp 10f
0B11:010C add dl, 37
0B11:010F mov ah, 02
0B11:0111 int 21
0B11:0113 int 20
0B11:0115
-g
F
Программа завершилась нормально

```

Загрузить число 4 в регистр CL и 5D в регистр DL:

```

-a 100
0B11:0100 shr dl, cl
0B11:0102
-r ip
IP 0100
:100
-t

AX=0000 BX=000F CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B11 ES=0B11 SS=0B11 CS=0B11 IP=0102  NV UP EI PL NZ NA PO NC
0B11:0102 80FA09          CMP     DL,09

```

ЛАБОРАТОРНАЯ РАБОТА №5

```

-a 100
0B07:0100 mov bp, 1f48
0B07:0103 mov ax, d892
0B07:0106
-t

AX=0000 BX=0000 CX=0000 DX=0000 SP=FFEE BP=1F48 SI=0000 DI=0000
DS=0B07 ES=0B07 SS=0B07 CS=0B07 IP=0103  NV UP EI PL NZ NA PO NC
0B07:0103 B892D8          MOV     AX,D892

```

```

-t
AX=D892 BX=0000 CX=0000 DX=0000 SP=FFEE BP=1F48 SI=0000 DI=0000
DS=0B07 ES=0B07 SS=0B07 CS=0B07 IP=0106  NU UP EI PL NZ NA PO NC
0B07:0106 D1D5          RCL     BP,1

```

Лабораторный практикум. Часть 7

Лекция

```

-a 100
0B0E:0100 mov dl, 41
0B0E:0102 mov cx, 000a
0B0E:0105 call 0200
0B0E:0108 loop 105
0B0E:010A int 20
0B0E:010C
-a 200
0B0E:0200 mov ah, 02
0B0E:0202 int 21
0B0E:0204 inc dl
0B0E:0206 ret
0B0E:0207
-g
*
л 8ф
Программа завершилась нормально

```

```

-r ip
IP 0102
:100
-a 100
0B0E:0100 call 200
0B0E:0103 int 20
0B0E:0105
-a 200
0B0E:0200 push cx
0B0E:0201 push dx
0B0E:0202 mov dl, 0a
0B0E:0204 call 300
0B0E:0207 inc dl
0B0E:0209 pop dx
0B0E:020A pop cx
0B0E:020B ret
0B0E:020C

```

Лабораторный практикум. Часть 8

Лекция

Ввод одной шестнадцатеричной цифры.

```

-a 100
0B11:0100 mov ah, 1
0B11:0102
-t
AX=0100 BX=0000 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B11 ES=0B11 SS=0B11 CS=0B11 IP=0102  NU UP EI PL NZ NA PO NC
0B11:0102 D583          AAD     83
-r ip
IP 0102
:100
-a 100
0B11:0100 int 21
0B11:0102
-g
3
Программа завершилась нормально

```

```

0B11:0206 mov dl, al
0B11:0208 cmp al, 30
0B11:020A jb 222
0B11:020C cmp al, 39
0B11:020E ja 215
0B11:0210 sub al, 30
0B11:0212 clc
0B11:0213 jmp 223
0B11:0215 cmp al, 41
0B11:0217 jb 222
0B11:0219 cmp al, 46
0B11:021B ja 222
0B11:021D sub al, 37
0B11:021F clc
0B11:0220 jmp 223
0B11:0222 stc
0B11:0223 jc 204
0B11:0225 mov bl, al
0B11:0227 mov ah, 2
0B11:0229 int 21
0B11:022B pop dx
0B11:022C pop ax
0B11:022D ret
0B11:022E

```

```

-a 100
0B11:0100 call 200
0B11:0103 int 20
0B11:0105
-g
F
Программа завершилась нормально

```

ЛАБОРАТОРНАЯ РАБОТА №6

```

-a 100
0B0E:0100 mov cx, 4
0B0E:0103 mov bx, 0
0B0E:0106 lodsb
0B0E:0107 cmp al, 30
0B0E:0109 cmp al, 39
0B0E:010B jg 170
0B0E:010D sub al, 30
0B0E:010F jmp 180
0B0E:0111 cmp al, 46
0B0E:0113 jg 190
0B0E:0115 sub al, 41
0B0E:0117 add al, 0a
0B0E:0119 shl bx, 4

0B0E:0119 add bx, ax
0B0E:011B loop 156
0B0E:011D ret
0B0E:011E mov ah, 4c
0B0E:0120 int 21
0B0E:0122

```

```

-a 200
0B0E:0200 mov cx, 10
0B0E:0203 mov dx, [sil
0B0E:0205 shl dx, 1
0B0E:0207 jc 220
0B0E:0209 mov dl, 30
0B0E:020B jmp 230
0B0E:020D mov dl, 31
0B0E:020F mov ah, 2
0B0E:0211 int 21
0B0E:0213 loop 203
0B0E:0215 ret
0B0E:0216

```

Адрес: е 120

0120: 56 76 65 64 69 74 65 20 73 6F 64 65 72 6A 69 6D

012E: 6F 65 20 72 65 67 69 73 74 72 61 20 41 58 3A 24

```

-e 138
0B11:0138 CD.28 21.41 72.58 39.29 8B.20 FA.3d 33.20 C0.24

-e 148
0B11:0148 C0.56 74.76 0C.65 32.64 E4.69 E8.74 23.65 E2.20
0B11:0150 74.73 F1.6f 47.64 FE.65 C4.72 EB.6a EC.69

-g
Uvedite soderjimoje regis<AX> = F46B
Uvedite sodejimoje regis<BX> = 5A0C
AX=1111010001101011
BX=0101101000001100

```

Лабораторный практикум. Часть 9

ЛАБОРАТОРНАЯ РАБОТА №7

Часть 1

1. Создание каталога proba в корневом каталоге логического диска C:

```

C:\Documents and Settings\Администратор>cd c:\
C:\>md proba
C:\>cd proba
C:\proba>_

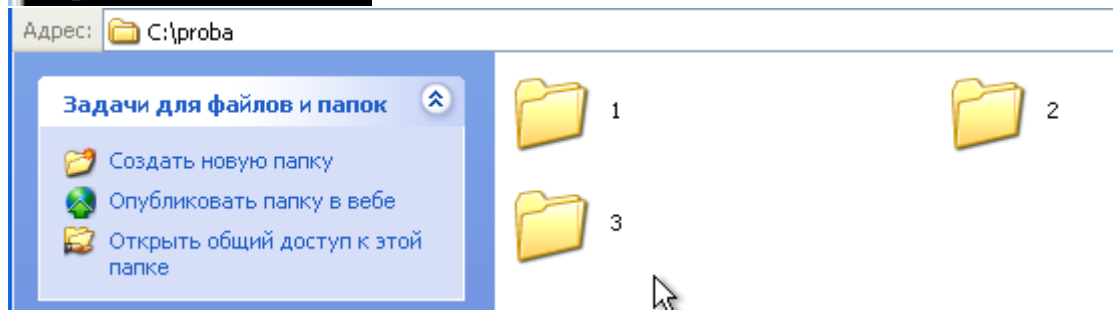
```

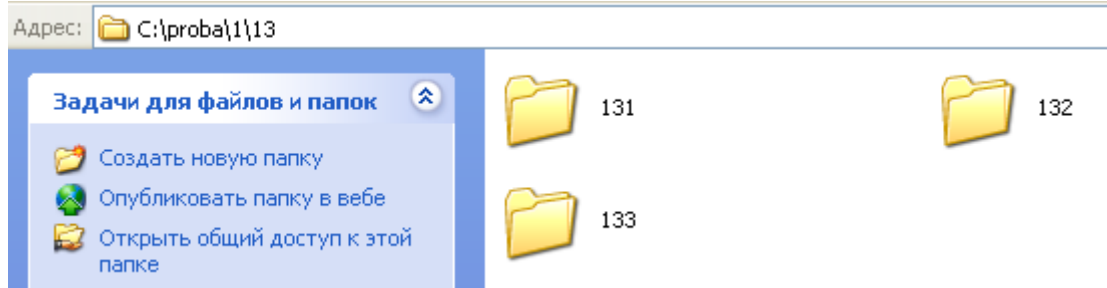
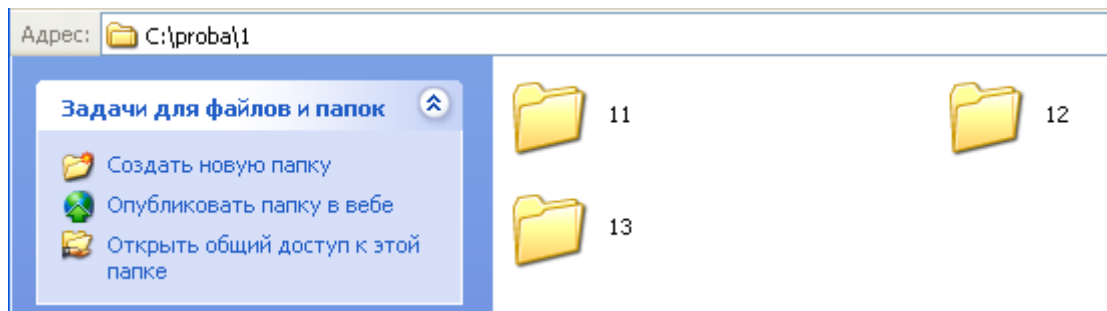
2. Создать дерево каталогов с корнем proba:

```

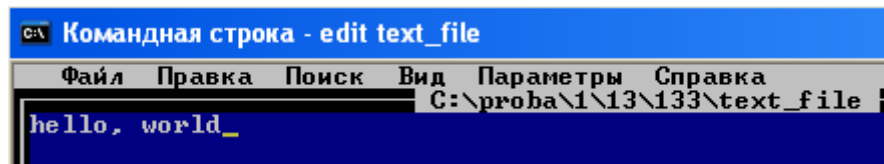
C:\proba>md 1
C:\proba>md 2
C:\proba>md 3
C:\proba>cd 1
C:\proba\1>md 11
C:\proba\1>md 12
C:\proba\1>md 13
C:\proba\1>cd 13
C:\proba\1\13>md 131
C:\proba\1\13>md 132
C:\proba\1\13>md 133

```

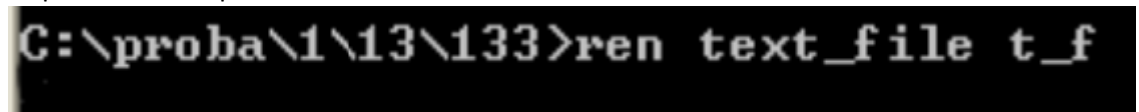




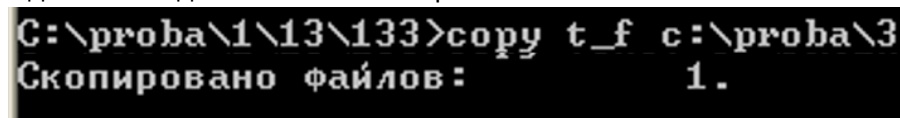
3. Создать текстовый файл, содержащий несколько слов русского или английского языка в каталоге 133.



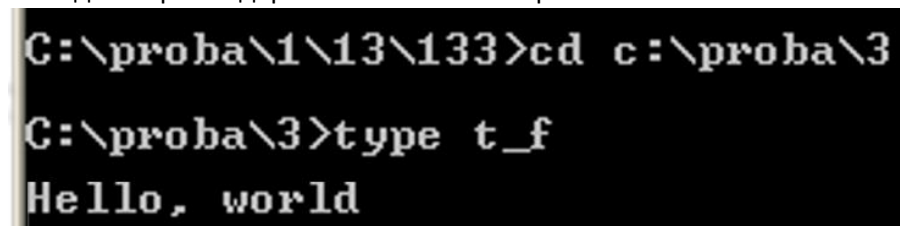
4. Переименовать файл.



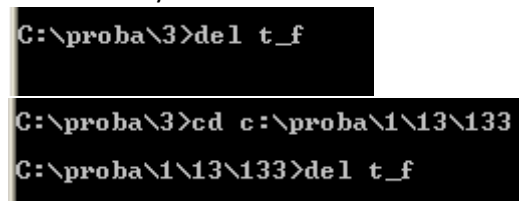
5. Одной командой выполнить копирование в каталог 3.



6. Вывод на экран содержимого текстового файла.



7. Выполнить уничтожение обоих текстовых файлов.



8. Поместить в каталог 2, а затем выполнить командный файл, содержащий команды, перечисленные в предыдущих шагах.

```
Файл  Правка  Поиск  Вид  Параметры  Справка
C:\proba\2\cmd_file.bat
edit text_file
ren text_file t_f
copy t_f c:\proba\3
cd c:\proba\3
type t_f
del t_f
cd c:\proba\1\13\133
del t_f
```

9. Выполнить уничтожение всех созданных файлов и каталогов.

```
C:\proba\1\13>rd 133
C:\proba\1\13>cd ..
C:\proba\1>rd 13
C:\proba\1>cd ..
C:\proba\1>rd 11
C:\proba\1>rd 12
C:\proba\1>cd 13
C:\proba\1\13>rd 131
C:\proba\1\13>rd 132
C:\proba\1>cd ..
C:\proba>rd 1
C:\proba>rd 2
Папка не пуста.
C:\proba>rd 3
C:\proba>cd ..
C:\>rd proba
```

Часть 2.

С темой была ознакомлена в курсе «Операционные системы». Информация понятно и усвоена.

Лабораторный практикум. Часть 10

ЛАБОРАТОРНАЯ РАБОТА №8

```
section .data
    hex_digit db '00$', 0

section .text
    global _start

_start:
    mov eax,4
    mov ebx,1
    mov ecx, hex_digit
    mov edx, 3 ;

    int 0x80

    mov eax,1
    mov ebx, ebx
    int 0x80
```

Compilation time: 0.14 sec, absolute running time: 0.16 sec, cpu time: 0.01 sec, memory peak: 6 Mb, absolute service time: 0.5 sec
00\$

Лабораторный практикум. Часть 11

ЛАБОРАТОРНАЯ РАБОТА №9

main.asm

```
section .data
    word_data dw 3039h

section .bss
section .text
    global _start
_start:
    mov dx, word_data
    call Test_write_word_dec
    call Test_write_word_hex
    mov eax, 60
    xor edi, edisyscall
Test_write_word_dec:
    mov dx, word_data
    call Write_word_hex
    ret
```

```
Test_write_word_hex:
    mov dx, word_data
    call Write_word_hex
    ret

#include "write_digit_hex.asm"
#include "write_char.asm"
#include "write_word_dec.asm"
#include "write_word_hex.asm"
#include "write_byte_hex.asm"
```

Write_word_dec.asm

```

section .text

write_word_dec:
    push cx
    push ax

    mov cx, 0
    mov ax, dx
    mov bx, 10

.divide_loop:
    xor dx, dx
    div bx
    push dx
    inc cx
    test ax, ax
    jnz .divide_loop

```

```

.print_loop:
    pop dx
    add dl, '0'
    call Write_char
    loop .print_loop

    pop ax
    pop cx
    ret

```

Write_word_hex.asm

```

section .text

Write_word_hex:
    push ax
    push dx

    mov al, dh
    call Write_byte_hex

    mov al, dl
    call Write_byte_hex

    pop dx
    pop ax
    ret

```

Write_byte_hex.asm

```

section .text

Write_byte_hex:
    push bx
    mov bx, 16

    mov ah, 0
    mov al, dl
    shr al, 4
    call Write_digit_hex

    mov al, dl
    and al, 0fh
    call Write_digit_hex

    pop bx
    ret

```

Write_digit_hex.asm

```

section .text

Write_digit_hex:
    cmp al, 10
    jb .digit
    add al, 'A' - 10
    jmp .done

.digit:
    add al, '0'

.done:
    call Write_char
    ret

```

Write_char.asm

```

section .text

Write_char:
    mov eax, 1
    mov edi, 1
    mov rsi, rsp
    mov edx, 1
    syscall
    ret

```

```

C:\Documents and Settings\Администратор>program.exe
12345
3039

```

Лабораторный практикум. Часть 12

ЛАБОРАТОРНАЯ РАБОТА №10

source.asm

```
%include "write_word.asm"
%include "write_byte.asm"
%include "write_char.asm"
%include "write_address.asm"
%include "write_digit.asm"

section .data
    word_data dw 3039h

section .bss

section .text
    global _start
```

```
_start:
    mov rdx, word_data
    call Test_write_word_dec
    movzx rdx, word_data
    call Test_write_word_hex

    mov eax, 60
    xor edi, edi
    sescall

Test_write_word_dec:
    movzx rdx, word_data
    call Write_word_dec
    ret

Test_write_word_hex:
    movzx rdx, word_data
    call Write_word_hex
    ret
```

Write_word.asm

```
%include "write_byte.asm"
%include "write_char.asm"
%include "write_address.asm"
```

```
Write_word_dec:
    push rcx
    push rax

    mov rcx, 0
    mov rax, rdx
    mov rbx, 10

.divide_loop:
    xcr rdx, rdx
    div rbx
    push rdx
    inc rcx
    test rax, rax
    jnr .divide_loop
```

```
.print_loop
    pop rdx
    add dl, '0'
    call Write_char
    loop .print_loop

    pop rax
    pop rcx
    ret
```

```
Write_word_hex:
    push rax
    push rdx

    movzx rax
    call Write_byte_hex
    movzx rax, dl
    call Write_byte_hex
```

```
movzx rax, dh
call Write_char_representation
movzx rax, dl
call Write_char_representation

pop rdx
pop rax
ret
```

Write_byte.asm

```

#include "write_digit.asm"
#include "write_char.asm"

Write_byte_hex:
    push rbx
    mov rbx, 16

    mov al, dl
    shr al, 4
    call Write_digit_hex

    mov al, dl
    and al, 0fh
    call Write_digit_hex

    pop rbx
    ret

```

Write_char.asm

```

#include "write_byte.asm"

Write_char:
    cmp al, 20h
    je .check_special_characters

    mov eax, 1
    mov edi, 1
    mov rsi, rsp
    mov edx, 1

```

```

C:\Documents and Settings\Администратор>program.exe
3039 41 42 43 44 45 46 47 00 20 1F 80 81 82 83 84 85 ABCDEFG. . 4T|+

```